

Living Earth Cube-in-a-Box - Admin Guide

Bruno Chatenoux

2026-03-05

Table of contents

1	Overview	3
2	Prerequisites	4
2.1	Linux	4
2.2	macOS	4
2.3	Windows (WSL2 Recommended)	4
2.4	Quick verification	5
3	Repository Structure	6
4	First-Time Setup	7
5	User Management	10
5.1	How User Authorization Works	10
5.2	Adding Authorized Users	10
5.3	User Signup Flow	11
5.4	Managing Existing Users	11
5.5	User Data and Notebooks	12
6	CiaB Management with <code>make</code>	13
6.1	Target reference (from <code>make help</code>)	13
6.2	Common usage patterns	13
6.3	STAC indexation configs	14
7	Execution Modes	16
7.1	Production Mode (<code>prod</code>)	16
7.2	Development Mode (<code>dev</code>)	16
7.3	Verification	17
8	Shared Directory	18
9	Backup and Restore	19
9.1	Database backup procedure	19
9.2	Volume backup procedure	19
10	Security Best Practices	21
11	License	22

1 Overview

Welcome to the Living Earth Cube-in-a-Box Admin Manual. This guide is intended for system administrators responsible for setting up, configuring, and maintaining the Open Data Cube (ODC) environment.

The Living Earth Cube-in-a-Box is a streamlined platform for running the Open Data Cube. It leverages Docker and GNU Make to simplify the deployment of a full stack including:

- **JupyterHub**: Multi-user server management that provides each user with a **JupyterLab** notebook environment.
- **PostgreSQL/PostGIS**: Database for ODC metadata.
- **Explorer**: Web UI for browsing datasets.
- **Traefik**: Reverse proxy and edge router.

The current repository is a fork of <https://github.com/opendatacube/cube-in-a-box> with the following major changes:

- Multi-user JupyterHub
- shared folders for collaboration
- Planetary Computer as a STAC datastore
- ODC Explorer
- Traefik integration as a reverse proxy
- Multi-architecture support (AMD64 & ARM64)
- Dask integration for parallel processing
- Admin and User documentation (Quarto)

All the developments have made possible thanks to the financial support of the European Union 'Horizon Europe Program' that funded the [LandShift](https://landshift.eu/)¹ (Grant Agreement no. 101182007), [Nos-tradamus](https://nostradamus-project.eu)² (Grant Agreement no. 101134888), and [NEMESIS](https://www.nemesis-soil.eu/)³ (Grant Agreement no. 101219087) projects.

¹<https://landshift.eu/>

²<https://nostradamus-project.eu>

³<https://www.nemesis-soil.eu/>

2 Prerequisites

This project is run via `docker compose` and a `Makefile`. Ensure your system meets the following requirements before starting:

2.1 Linux

- **Docker Engine:** [Official Instructions](#)¹.
- **Docker Compose:** Usually included with Docker Engine.
- **GNU Make:** Install via your package manager (e.g. `sudo apt update && sudo apt install make`).

2.2 macOS

- **Docker Desktop:** [Official Instructions](#)².
- **Make:** Available via Xcode Command Line Tools (`xcode-select --install`).

2.3 Windows (WSL2 Recommended)

1. **WSL2:** Install [WSL2](#)³ and a Linux distribution (e.g., Ubuntu).
2. **Docker Desktop:** Install [Docker Desktop for Windows](#)⁴. In settings, enable **Use WSL2 based engine** and **WSL Integration** for your Linux distribution.
3. **Make:** Open your WSL terminal and install Make (`sudo apt update && sudo apt install -y make`).

Notes

- Run all `make ...` commands **from the WSL terminal** (not PowerShell) to ensure a consistent Linux-like environment.
- Instead of keeping this project in your regular Windows folders (like your `C:` drive, which appears as `/mnt/c/...` in Linux), save it directly inside your Linux environment (for example, in a `~/projects/` folder). This will make the system run much faster.

¹<https://docs.docker.com/engine/install/>

²<https://docs.docker.com/desktop/setup/install/mac-install/>

³<https://learn.microsoft.com/en-us/windows/wsl/install>

⁴<https://docs.docker.com/desktop/setup/install/windows-install/>

2.4 Quick verification

Once installed, you should be able to run:

```
make --version  
docker --version  
docker compose version
```

3 Repository Structure

Core Files:

- `Makefile`: Main entry point for commands (setup, indexing, maintenance).
- `docker-compose.yml` / `docker-compose.dev.yml`: Service definitions for prod and dev environments.
- `.env.default` / `.env`: Environment variables for system configuration.
- `Dockerfile` / `docker-bake.hcl`: Definitions for building the Docker images.
- `index-from-config.sh` / `product-from-config.sh`: Config-driven STAC indexing and ODC product registration.
- `config/indexation/`: YAML preset for `INDEX_CONFIG` (`default.yaml`).
- `requirements.txt`: Python package dependencies for the Jupyter environment.
- `distributed.yaml`: Configuration for distributed Dask processing.

Key Directories:

- `data/`: Database storage, local files, and user workspaces.
- `datacube-explorer/`: Web UI configuration and customization.
- `hub/`: JupyterHub setup, authentication, and Docker spawner logic.
- `config/indexation/`: STAC indexation preset (`default.yaml`, `INDEX_CONFIG`).
- `config/products/`: ODC product YAMLs by source (`pc/`, `cop/`).
- `quarto/`: Documentation source (including this manual).
- `scripts/`: Data processing and system maintenance utilities.
- `shared/`: Read-only templates and shared data for all JupyterHub users.

4 First-Time Setup

! Execution Context

All admin operations in this chapter must be executed from the **project root directory** (where the `Makefile` and `docker-compose.yml` reside).

This repository uses environment variables to configure the local domain, database credentials, and the Jupyter password.

1. Create a `.env` file (Docker Compose reads `.env` by default):

```
cp .env.default .env
```

2. Edit `.env` to match your setup:

- Set `DOMAIN` to your local domain (e.g. `localhost` or a IP)
- Set strong passwords for `POSTGRES_PASS`
- Configure `JUPYTERHUB_ADMINS` (**with at least one admin username**)
- Optionally add regular users to `JUPYTERHUB_USERS`

Required variables:

Table 4.1: Required environment variables

Variable	Required	Example	Description
<code>DOMAIN</code>	Yes	<code>localhost</code>	Hostname used to access the web endpoints (Jupyter/Explorer).
<code>IMAGE_VERSION</code>	Yes	<code>20260211</code>	Version of the images to use.
<code>POSTGRES_HOSTNAME</code>	Yes	<code>postgres</code>	Hostname used to access the PostgreSQL database.
<code>POSTGRES_PORT</code>	Yes	<code>5432</code>	Port used to access the PostgreSQL database.
<code>POSTGRES_DBNAME</code>	Yes	<code>opendatacube</code>	PostgreSQL database name used by Open Data Cube.
<code>POSTGRES_USER</code>	Yes	<code>opendatacube</code>	PostgreSQL user for the Open Data Cube database.
<code>POSTGRES_PASS</code>	Yes	<code>a-strong-password</code>	PostgreSQL password for the Open Data Cube database.
<code>JUPYTERHUB_ADMINS</code>	Yes	<code>admin,bruno</code>	Comma-separated list of JupyterHub admin usernames.
<code>JUPYTERHUB_USERS</code>	No	<code>guest,alice,bob</code>	Comma-separated list of authorized non-admin usernames.
<code>CDSE_S3_ACCESS_KEY</code>	No	<code>(empty)</code>	Copernicus Data Space S3 access key (required for CDSE-indexed products).
<code>CDSE_S3_SECRET_KEY</code>	No	<code>(empty)</code>	Copernicus Data Space S3 secret key.
<code>CDSE_S3_ENDPOINT</code>	No	<code>https://eodata.dataspace.copernicus.eu</code>	CDSE S3 API endpoint.

3. First-time setup

- Using default parameters:

```
make setup
```

- With a specific area/time (`BBOX`, `DATETIME`):

```
# Switzerland 1 year
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31

# Switzerland all years (till end 2025, might take a while)
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=1984-01-01/2025-12-31
```

Table 4.2: `make setup` parameters

Parameter	Format	Description
<code>BBOX</code>	<code>lon_min,lat_min,lon_max,lat_max</code>	Spatial extent in decimal degrees. Use a tool like bboxfinder.com ¹ (copy the Box values).
<code>DATETIME</code>	<code>start/end</code>	Temporal extent. Use YYYY-MM-DD format (e.g., 2020-01-01/2020-12-31).
<code>INDEX_CONFIG</code>	path to YAML	STAC indexation preset (default: <code>config/indexation/default.yaml</code>).

! Copernicus Data Space products

`config/indexation/default.yaml` indexes Sentinel-2 (`s2_l2a_cdse`) and CLCplus from Copernicus Data Space STAC², plus Planetary Computer products (including `s2_l2a_pc`).

- Set `CDSE_S3_ACCESS_KEY` and `CDSE_S3_SECRET_KEY` in `.env` ([S3 credentials](#)³). These keys are required for **indexing**, **Explorer downloads**, and **notebook access** to CDSE products. Explorer download links are signed automatically (no proxy secret to configure); notebooks read CDSE data directly via `s3://` and do not use that signing path.
- **S3 endpoint pinning:** when `CDSE_S3_ACCESS_KEY`/`CDSE_S3_SECRET_KEY` are set, notebook and Explorer S3 access is pinned to the CDSE endpoint (`AWS_S3_ENDPOINT=eodata...`, `AWS_VIRTUAL_HOSTING=FALSE`). Generic `s3://` reads from other providers (e.g. AWS-hosted buckets) will not work in that configuration; Planetary Computer assets are unaffected (served over HTTPS with SAS tokens).
- CLCplus is **Europe-only** — use a European `BBOX` (e.g. Switzerland below).
- If CDSE indexing fails with transient errors (e.g. 504 or STAC API timeouts), retry with `make index-serie (PARALLELISM=1)` for a gentler load on the catalog. This is a mitigation for server flakiness, not evidence of access limits.
- In notebooks, use `get_patch_url` from `utils.le_dc` (CDSE access uses `s3://` paths with GDAL header auth via `utils/le_cdse_s3.py`; presigned URLs are not supported by CDSE):

```
from utils.le_dc import get_patch_url

ds = dc.load(..., patch_url=get_patch_url(dc, product))
```

Copying `notebooks_demo/` to another location includes `utils/le_cdse_s3.py`; no separate signing module is required. Ensure `CDSE_S3_ACCESS_KEY` and `CDSE_S3_SECRET_KEY` are set, then restart your Jupyter server so Dask workers inherit the credentials.

¹<http://bboxfinder.com>

³

<https://stac.dataspace.copernicus.eu/v1/>

³

- European setup example (CLCplus):

```
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31
```

Note

`make product` and `make index` both use `INDEX_CONFIG` (default: `config/indexation/default.yaml`). Only products referenced via `product_file` in that config are registered and indexed. Sentinel-2 L2A is split into `s2_l2a_pc` (Planetary Computer) and `s2_l2a_cdse` (CDSE); other ODC names are unchanged (`nasadem`, etc.). Source is recorded in each product YAML under `metadata.product.source`. List larger or slower products first in the config `products` list so parallel indexing starts them before smaller ones.

5 User Management

JupyterHub uses NativeAuthenticator with a custom signup handler that restricts access to pre-authorized users only.

But admin users can add new users through the JupyterHub admin panel.

5.1 How User Authorization Works

1. **Authorized Users:** Only users listed in `JUPYTERHUB_ADMINS` or `JUPYTERHUB_USERS` in the `.env` file (or [manually added by an admin user](#)) can successfully sign up.
2. **Unauthorized Users:** Unauthorized users will see an error message directing them to contact the administrator if they try to sign up.
3. **Self-Service Signup:** Authorized users can create their own accounts via the signup page.
4. **Admin Creation:** Administrators can also create user accounts through the JupyterHub admin panel.

5.2 Adding Authorized Users

Method 1: Via `.env` file (Recommended for initial setup)

Edit the `.env` file:

```
# Admin users (have full control over JupyterHub)
JUPYTERHUB_ADMINS=admin,bob

# Regular users (can only access their own notebooks)
JUPYTERHUB_USERS=guest,charlie
```

Notes

- You need to have at least one admin user (`JUPYTERHUB_ADMINS`)!
- If you add a user this way while the system is running, you need to restart JupyterHub to apply changes:

```
docker-compose restart jupyterhub
```

Users can now visit <http://<DOMAIN>/jupyter/hub/signup> to create their accounts.

Method 2: Via JupyterHub Admin Panel (Once the system is running, for ad-hoc user additions)

1. Log in as an admin user.

2. **File** > **Hub Control Panel** > **Admin**, or navigate to <http://<DOMAIN>/jupyter/hub/admin>.
3. Click **Add Users**.
4. Enter the username and click **Add**.
5. The user is created immediately and can sign up at <http://<DOMAIN>/jupyter/hub/signup>.

5.3 User Signup Flow

For Authorized Users:

1. Visit <http://<DOMAIN>/jupyter/hub/signup>.
2. Fill in username (must match an authorized one) and password.
3. Submit the form.
4. See success message: "The signup was successful! You can now go to the home page and log in to the system."
5. Log in at <http://<DOMAIN>/jupyter/hub/login>.

For Unauthorized Users:

1. Contact the administrator to be added.

5.4 Managing Existing Users

View all users:

1. Log in as admin.
2. Navigate to <http://<DOMAIN>/jupyter/hub/admin>.
3. You'll see a list of all users with their status and last activity.

Edit user:

1. Click "Stop Server" and then "Edit User" next to any user.
2. You can make them admin, delete them, or manage their servers.

Delete user:

1. Click "Edit User" □ "Delete User".
2. This removes the user account but doesn't delete their notebook files (stored in [jupyterhub-user-<username>](#) volume).
3. Execute `make purge-user HUB_USER=<username> CONFIRM=1` to delete the user's volume.
4. Remove user from `.env` file if required.

Password reset or recovery:

1. Using the JupyterHub Admin interface, delete the user and re-create them, **without deleting their user volume**.
2. Inform the user they will have to sign up again.

5.5 User Data and Notebooks

Each user's notebooks are stored in a Docker volume named `jupyterhub-user-<username>`. These volumes persist even if the user account is deleted.

Backup user notebooks:

```
docker run --rm -v jupyterhub-user-<username>:/source -v $(pwd)/backups:/backup  
↪ alpine tar czf /backup/user-<username>-notebooks.tar.gz -C /source .
```

Restore user notebooks:

```
docker run --rm -v jupyterhub-user-<username>:/target -v $(pwd)/backups:/backup  
↪ alpine tar xzf /backup/user-<username>-notebooks.tar.gz -C /target
```

Remove all user volumes (⚠️⚠️ use with extra care):

```
make purge-users CONFIRM=1 # ⚠️⚠️
```

Remove a specific user volume (⚠️ use with care):

```
make purge-user HUB_USER=alice CONFIRM=1 # ⚠️
```

6 CiaB Management with `make`

The system uses `GNU Make`¹ as a task automation tool to wrap complex Docker and data indexing workflows into simple commands. Instead of manually executing long command-line sequences, you use `make` targets to trigger predefined administrative tasks.

To see the full list of available targets on your machine, run:

```
make help
```

6.1 Target reference (from `make help`)

Table 6.1: `make` target reference

Command	Description
Runtime Control	
<code>make up</code>	Start the environment in the background (then open Jupyter in your browser)
<code>make down</code>	Stop the running services (keeps your data and images)
<code>make status</code>	Show what is running (containers and their status)
<code>make logs</code>	Show live logs from all services (useful for troubleshooting)
<code>make shell</code>	Open a terminal inside the Jupyter container (requires <code>HUB_USER</code>)
<code>make wait-for-db</code>	Wait for PostgreSQL to be ready to accept connections
Setup & Init	
<code>make setup</code>	First-time setup (mode-dependent: uses pull in <code>prod</code> , build in <code>dev</code>)
<code>make init</code>	Initialize the Open Data Cube database (run once after <code>setup</code>)
<code>make build</code>	Build the images locally
<code>make build-nocache</code>	Build the images locally from scratch
<code>make pull</code>	Download all service images (recommended before first run in <code>prod</code> mode)
Data & Indexing	
<code>make product</code>	Load ODC product definitions from <code>INDEX_CONFIG</code> into the database
<code>make index</code>	Index products from <code>INDEX_CONFIG</code> (parallel; uses <code>BBOX</code> , <code>DATETIME</code>)
<code>make index-parallel</code>	Same as <code>make index</code>
<code>make index-serie</code>	Index products from <code>INDEX_CONFIG</code> sequentially (<code>PARALLELISM=1</code>)
<code>make update-explorer</code>	Rebuild the Explorer index so datasets appear in the web UI
Maintenance	
<code>make backup</code>	Create a backup of the PostgreSQL database
<code>make restore</code>	Restore PostgreSQL database from a backup file (requires <code>BACKUP_FILE</code> and <code>CONFIRM=1</code>)
<code>make clean</code>	Stop everything and remove containers, volumes, and built images
<code>make purge-data</code>	Delete local data in <code>./data</code> (pg, local_data, shared). Irreversible; requires <code>CONFIRM=1</code>
<code>make purge-user</code>	Remove a specific user container and volume. Irreversible; requires <code>HUB_USER</code> and <code>CONFIRM=1</code>
<code>make purge-users</code>	Remove all spawned JupyterHub user containers and volumes. Irreversible; requires <code>CONFIRM=1</code>
Advanced	
<code>make release-push</code>	Build and push multi-architecture production images to the configured container registry
<code>make help</code>	Show available commands

6.2 Common usage patterns

- First-time setup (default parameters):

¹<https://www.gnu.org/software/make/>

```
make setup
```

- Setup with a specific area/time (**BBOX** and **DATETIME**):

```
# Switzerland 1 year
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31

# Switzerland all years (till end 2025, might take a while)
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=1984-01-01/2025-12-31
```

- Start/stop and troubleshoot:

```
make up
make status
make logs
make down
```

- Reset options (⚠ use with care):

```
# Stop everything and remove containers/volumes/images
make clean

# ⚠ Irreversible: delete local data in ./data (requires confirmation)
make purge-data CONFIRM=1

# ⚠⚠ TOTAL WIPE OUT (USE WITH EXTRA CARE !!!)
make clean && make purge-data CONFIRM=1 && make purge-users CONFIRM=1
# then check for any remaining resources
docker ps -a && docker images -a && docker volume ls && ls -la ./data
```

6.3 STAC indexation configs

make product and **make index** both read the same **INDEX_CONFIG** YAML. Only products listed in that file are registered and indexed.

Indexation presets live in **config/indexation/**. ODC product definitions live in **config/products/pc/** (Planetary Computer) and **config/products/cop/** (Copernicus CDSE), with **metadata.product.source** set in each YAML.

List larger or slower products first in the **products** section of **INDEX_CONFIG** so parallel **make index** starts them before smaller ones (see **config/indexation/README.md**).

Config	Source(s)	Products
default.yaml (default)	Planetary Computer + CDSE	11 products: s2_12a_pc , s2_12a_cdse , CLCplus (CDSE); S1 RTC, Landsat, NASADEM, ESA WorldCover, ESRI land cover (PC)

```
# Default (all products)
make index BBOX=25,20,35,30 DATETIME=2021-12-01/2021-12-31

# Per-product timing report at the end
make index TIME_INDEX=1 BBOX=25,20,35,30 DATETIME=2021-12-01/2021-12-31
```

```
# European BBOX recommended for CLCplus
make index BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31

# Sequential indexing (gentler on flaky catalogs such as CDSE)
make index-serie BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31
```

If CDSE indexing fails with transient errors (e.g. 504 or STAC API timeouts), retry with `make index-serie (PARALLELISM=1)`. This reduces concurrent load on the catalog; it is a mitigation for server flakiness, not evidence of access limits.

7 Execution Modes

Living Earth Cube-in-a-Box supports two execution modes: **Production** (`prod`) and **Development** (`dev`). The `MODE` environment variable determines which Docker Compose configuration is used.

Table 7.1: Execution modes comparison

Mode	Purpose	Image Source	Compose Files
Production (<code>prod</code>)	Standard deployment (default).	Pre-built remote images.	<code>docker-compose.yml</code>
Development (<code>dev</code>)	Local testing and modifications.	Locally built images (<code>:dev</code>).	<code>+ docker-compose.dev.yml</code>

7.1 Production Mode (`prod`)

This is the default mode. It pulls official images from the registry and is optimized for stability.

```
# Explicitly set (optional)
export MODE=prod
make up
```

7.2 Development Mode (`dev`)

Use this mode to modify Dockerfiles or core configurations. In this mode, `make setup` builds images locally instead of pulling them.

Method 1: Session-wide (Recommended)

This ensures all subsequent `make` commands (`up`, `status`, `logs`, etc.) use the correct mode.

```
export MODE=dev
make setup
make up
```

Method 2: One-off Command

You can pass the variable directly, but you must remember to include it in **every** command.

```
make up MODE=dev
make status MODE=dev
```

Switching Back

To return to production mode:


```
unset MODE  
# or  
export MODE=prod
```

7.3 Verification

Run `make help` to verify the active mode:

```
Current mode: dev  
...
```

8 Shared Directory

This directory is shared among users of the JupyterHub instance.

The primary purpose of this directory is to facilitate file sharing and collaboration between users.

The `./shared/` directory contains:

- **Static Content:** Directories and files directly under the root are read-only for everyone in the Jupyter interface.
- **User Directories:** Directories and files under `./shared/all_users/` are visible to all users in the Jupyter interface as read-only, with only their own directory as read-write. This allows users to copy notebooks from others but not modify their work directly.

Note

CiaB Admin can add/remove/modify `./shared/` directory content at any time on the host machine. The changes will be visible to all users in the Jupyter interface immediately.

9 Backup and Restore

The `./data/` subdirectories contain persistent data and should be backed up regularly:

- `./data/pg/` - PostgreSQL database files
- `./data/local_data/` - Local data cache
- `./data/jupyterhub_data/` - JupyterHub configuration and user data

User notebooks are stored in Docker volumes named `jupyterhub-user-<username>`

This chapter describes how to backup and restore the CiaB Database and user volumes to and from the `./backups/` directory.

9.1 Database backup procedure

Database backup:

To create a backup of your PostgreSQL database:

```
make backup
```

This will create a timestamped SQL dump file in the `./backups` directory.

Example: `./backups/opendatacube_20260121_141530.sql`.

Restore Database:

To restore a database from a backup file:

```
make restore BACKUP_FILE=./backups/opendatacube_20260121_141530.sql CONFIRM=1
```

⚠ Notes

Restoring will overwrite your current database. Make sure you have a recent backup before proceeding.

9.2 Volume backup procedure

Manual volume backup:

```
# Backup user notebooks
docker run --rm -v jupyterhub-user-<username>:/source -v $(pwd)/backups:/backup
↪ alpine tar czf /backup/user-<username>-notebooks.tar.gz -C /source .

# Backup all data directories
tar czf backups/data-backup-$(date +%Y%m%d).tar.gz ./data/
```

Restore user notebooks:

```
# Restore user notebooks
docker run --rm -v jupyterhub-user-<username>:/target -v $(pwd)/backups:/backup
↳ alpine tar xzf /backup/user-<username>-notebooks.tar.gz -C /target
```

10 Security Best Practices

1. **Use strong passwords** for admin accounts
2. **Regularly review** the user list in the admin panel
3. **Remove unused accounts** to minimize security risks
4. **Backup user data** regularly (see Backup and Restore section)
5. **Keep admin users minimal** - only trusted users should have admin access

11 License

This project is a fork of <https://github.com/opendatacube/cube-in-a-box>, and consequently inherits its **MIT License**.



Figure 11.1: License: MIT

Copyright (c) 2018 Alex Leith
Copyright © 2026 UNIGE/GRID-Geneva

You are free to use, modify, and distribute this software under the terms of the MIT License. For more details, see the full license text: [MIT](https://opensource.org/licenses/mit)¹.

¹<https://opensource.org/licenses/mit>